

Скольльзящий максимум на Vitis HLS (ДЗ, лекция 15)

IP-блок скользящего максимума, написанный на C++ и синтезированный в Vitis HLS 2025.2 для кристалла xc7z020clg400-1 (такт 10 нс). Блок проверен C-тестбенчем, Co-simulation и Verilog-тестбенчем в Vivado.

Текст задания: [task/task.txt](#).

Что делает блок

Функция `void moving_max(int in_data[64], int out_data[64])` для каждого отсчёта выдаёт наибольшее из последних восьми входных значений, включая текущее:

```
out_data[n] = max(in_data[n-7] ... in_data[n])
```

Для первых семи отсчётов ($n < 7$) максимум берётся по имеющимся: от `in_data[0]` до `in_data[n]`.

Реализация — сдвиговый регистр `window` на 8 элементов. На каждом шаге окно сдвигается, новый отсчёт кладётся в `window[0]`, затем ищется максимум по окну. Перед началом окно заполняется `INT_MIN`, поэтому «пустые» ячейки на результат не влияют. Каждый входной элемент читается в коде один раз; `malloc` и фиксированные адреса не используются.

Размеры заданы макросами `N_SAMPLES` (64) и `WINDOW` (8) в `moving_max.h`.

Состав проекта

Путь	Что это
<code>vitis/example/moving_max.cpp</code> , <code>moving_max.h</code>	Функция (текущая версия — с <code>#pragma HLS PIPELINE II=1</code>)
<code>vitis/example/moving_max_tb.cpp</code>	C-тестбенч
<code>vitis/moving_max_test/</code>	HLS-компонент; настройки в <code>hls_config.cfg</code>
<code>vivado/moving_max_test/</code>	Проект Vivado с экземпляром IP <code>moving_max_0</code>
<code>vivado/moving_max_test/moving_max_test.srcs/sim_1/new/tb_moving_max.v</code>	Verilog-тестбенч

Проверка на C

Тестбенч сравнивает функцию с эталоном — прямым перебором по правилу из задания — на шести входных векторах и возвращает 0 при успехе, 1 при ошибке:

№	Вектор	Что проверяет
0	Возрастающая	Максимум — всегда текущий

№	Вектор	Что проверяет
	последовательность	отсчёт
1	Убывающая последовательность	Максимум — самый старый отсчёт в окне
2	Константа	Одинаковые значения
3	Одиночные импульсы	Максимум держится ровно 8 отсчётов
4	INT_MAX и INT_MIN на входе	Границы типа; INT_MIN не путается с пустой ячейкой
5	Псевдослучайные числа обоих знаков	Общий случай

Результат C Simulation: TEST PASSED: 6 vectors, 64 samples each.

Результаты C Synthesis

Собраны две версии функции. Автоконвейеризация циклов в обоих случаях выключена (`syn.compile.pipeline_loops=0`), так что разница — только в директиве.

	Без директив	#pragma HLS PIPELINE II=1 на MAIN_LOOP
Латентность, тактов	2058	79
Латентность, нс	20 580	790
Интервал между отсчётами, тактов	32	1
DSP	0	0
BRAM	0	0
FF	111	576
LUT	322	746
Оценка периода такта, нс	5,57	6,50

Что из этого следует:

- **Без директив** схема последовательная: сдвиг окна и поиск максимума — два цикла по 7 итераций, каждая по 2 такта, итого 32 такта на отсчёт. Один компаратор работает по очереди, поэтому логики мало.
- **С конвейером** HLS полностью разворачивает оба внутренних цикла: окно становится восемью регистрами, поиск максимума — деревом из семи компараторов. Блок принимает отсчёт каждый такт; глубина конвейера — 5 тактов.
- **Цена:** скорость выросла в 26 раз, FF — примерно в 5 раз, LUT — в 2,3 раза. DSP не нужны ни в одной версии: в алгоритме только сравнения.

Co-simulation и Package

Для конвейерной версии Co-simulation (Verilog, XSim) завершилась со статусом **Pass**: латентность 77 тактов, интервал между запусками 78 тактов, всего 467 тактов на шесть запусков. Отчёт синтеза даёт 79 тактов — это оценка сверху, реальный RTL на 2 такта короче.

IP упакован в формате IP Catalog (xilinx.com:hls:moving_max:1.0) и подключён в проект Vivado как `moving_max_0`.

Интерфейс блока

Директив интерфейса в коде нет, поэтому HLS сделал порты по умолчанию:

Порты	Назначение
<code>ap_clk</code> , <code>ap_rst</code>	Такт и синхронный сброс (активный уровень высокий)
<code>ap_start</code> , <code>ap_done</code> , <code>ap_idle</code> , <code>ap_ready</code>	Управление <code>ap_ctrl_hs</code>
<code>in_data_address0[5:0]</code> , <code>in_data_ce0</code> , <code>in_data_q0[31:0]</code>	Чтение входного массива из внешней памяти
<code>out_data_address0[5:0]</code> , <code>out_data_ce0</code> , <code>out_data_we0</code> , <code>out_data_d0[31:0]</code>	Запись результата во внешнюю память

Памяти внутри IP нет: блок сам выставляет адреса, а память подключает тот, кто его использует.

Симуляция в Vivado

Verilog-тестбенч `tb_moving_max.v` моделирует обе памяти (входную — с задержкой чтения в один такт), запускает блок по `ap_start` и сравнивает результат с эталоном на тех же шести векторах, что и C-тестбенч. Дополнительно проверяется, что блок сделал ровно 64 записи за запуск.

Результат Behavioral Simulation: **TEST PASSED: 6 vectors, 64 samples each**, латентность 77 тактов на каждом векторе, весь прогон — 4865 нс.

Ход одного запуска (первый вектор; фронты такта в 5, 15, 25 нс и далее):

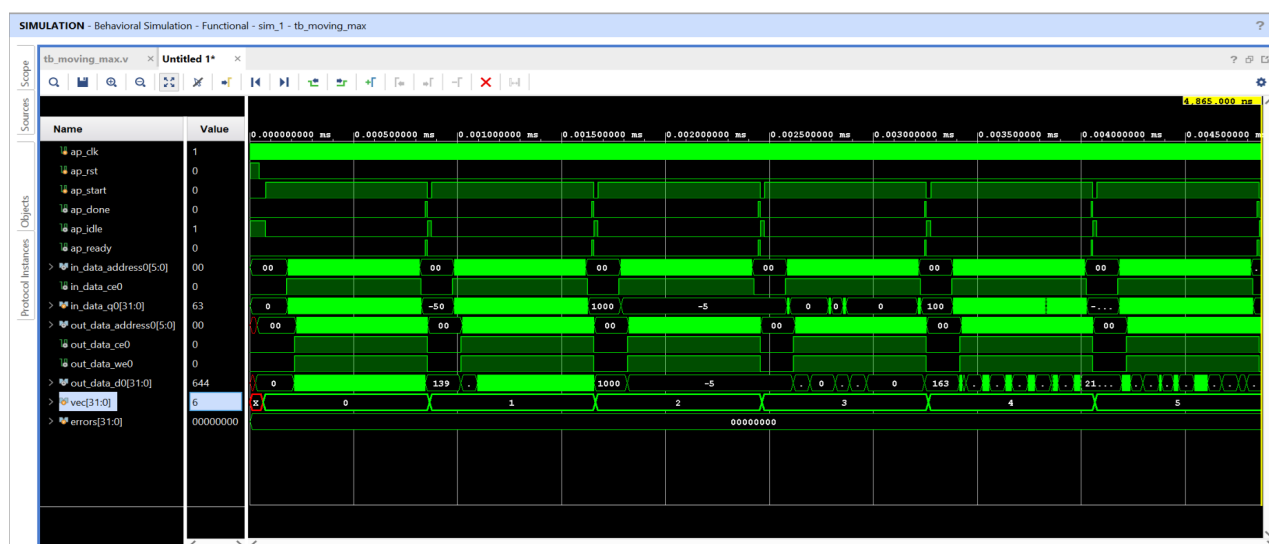
Время, нс	Событие
0–45	Сброс
75	<code>ap_start = 1</code> , <code>ap_idle</code> падает в 0
75–175	Заполнение окна значением <code>INT_MIN</code> , на портах памяти тишина
175	Начало чтения: <code>in_data_ce0 = 1</code> , адрес растёт на 1 каждый такт (<code>II = 1</code>)

Время, нс	Событие
215	Первая запись результата; запись отстаёт от чтения на 4 такта
845	Последняя запись (адрес 63), ap_done = ap_ready = 1 на один такт
855	ap_start = 0, ap_idle = 1

На векторе с импульсами (2475–3255 нс) на входе $in[0] = 100$, $in[20] = 120$, $in[27] = 127$, $in[63] = 163$, остальное — нули. На выходе: адреса 0–7 — 100, 8–19 — 0, 20–26 — 120, 27–34 — 127, 35–62 — 0, 63 — 163. Импульс 120 держится только 7 отсчётов, потому что на 27-м его перебивает больший.

Скриншот симуляции

Весь прогон, 0–4865 нс, шесть запусков блока подряд:

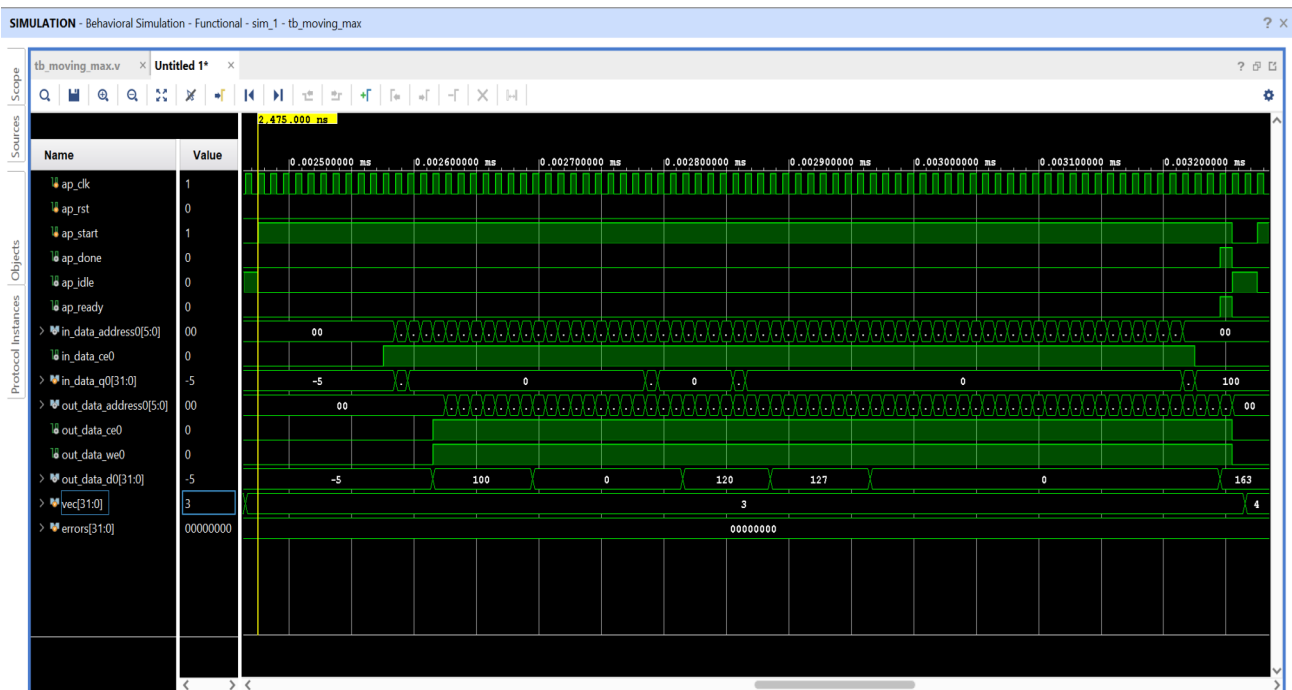


Что видно на скриншоте:

- **vec** — номер входного вектора, от 0 до 5; каждый запуск занимает 800 нс.
- **ap_start** держится весь запуск, **ap_done** и **ap_ready** — короткие импульсы в конце каждого запуска, **ap_idle** поднимается между запусками.
- **in_data_ce0** и **out_data_we0** активны почти весь запуск: блок читает и пишет одновременно, запись начинается на 4 такта позже чтения.
- **in_data_q0** и **out_data_d0** показаны в знаковом десятичном виде. На векторе 2 (константа) на обеих шинах стоит -5 ; значения 139, 1000 и 163 в конце запусков 0, 1 и 3 — это последние результаты, **out_data[63]**.
- **errors** остаётся равным 0 до конца прогона.

Сплошная заливка на шинах адреса и данных означает, что значение меняется каждый такт и в этом масштабе не помещается.

Крупный план четвёртого запуска ($vec = 3$, одиночные импульсы), 2475–3255 нс; курсор стоит на моменте подъёма `ap_start`:



Что видно на скриншоте:

- На `in_data_q0` импульсы 100, 120, 127 и 163 делятся по одному такту (узкие метки между нулями), а на `out_data_d0` они растянуты: 100 и 127 держатся по 8 тактов, 120 — 7 тактов, потому что его перебивает 127.
- Адреса `in_data_address0` и `out_data_address0` меняются каждый такт — интервал $\Pi = 1$.
- `in_data_ce0` поднимается через 10 тактов после `ap_start` (заполнение окна), `out_data_we0` — ещё на 4 такта позже (глубина конвейера).
- Последняя запись, 163 по адресу 63, совпадает с импульсом `ap_done` / `ap_ready`; после него `ap_idle` = 1 и начинается следующий запуск ($vec = 4$).
- Значение -5 на шинах данных в начале осталось от предыдущего запуска, а 100 на `in_data_q0` в конце — от холостого чтения адреса 0 после адреса 63; ни то, ни другое в результат не попадает.

Как повторить

1. Открыть папку `vitis/` в Vitis Unified IDE, компонент `moving_max_test`.
2. В панели *Flow* по очереди запустить C Simulation, C Synthesis, C/RTL Cosimulation, Package.
3. Для версии без директив убрать из `moving_max.cpp` строку `#pragma HLS PIPELINE II=1` и повторить C Synthesis.

4. Открыть `vivado/moving_max_test/moving_max_test.xpr`, запустить *Run Behavioral Simulation* и выполнить `run all` в Tcl-консоли.